



STUDY GUIDE · BRIEF EDITION · HIP OPERATIONS

Diagnosing incidents with AI on top of existing rules

What the use case is, why we need it, the real data behind it,
how the HIP platform and its logs work, what we designed and built,
what worked and what did not, and how to make the case to the client.

S1 Large-mapping heap

S2 PI7 listener hang

S3 Vendor release

S4 Stalled exchange

S5 Shared cache

AT A GLANCE

What this is, in one page

The use case

The client already runs 300 to 400+ monitoring rules and they fire correctly. What is slow is **diagnosis**: the hours between "alerts fired" and "we agree what broke."

This proof of concept shows an AI layer that **reads what the rules already reported**, plus the records an engineer would look at around them, and produces one timeline, ranked causes with evidence for and against, ordered checks, the blast radius and a recovery that only a named human can approve. It **explains; it does not detect**, replace any rule, or act on its own.

3,118 incidents, Jun to Aug 2026	41% raised automatically by rules	12.5% linked duplicates of another ticket	6.6 h median time an auto-raised ticket stays open
--	---	---	--

Alerting is healthy, one event raises several tickets, and tickets still stay open for hours. **Detection is not the bottleneck; diagnosis is.**

CONTENTS

1 The use case, and why we need it	3
2 How HIP works: one message, end to end	4
3 The real data we studied	5
4 Glossary: every term you will meet	6
5 The five scenarios	7
6 What we built	9
7 The argument: "We already have 300 to 400 rules"	10
8 Findings: what worked, what did not	11
9 Revision	12

How to read this brief

Sections 1 to 3 give the problem, the platform and the data. Keep **section 4 (glossary)** open as you read. Sections 5 and 6 cover the scenarios and what was built. Section 7 is the argument for the client, section 8 the honest findings, section 9 revision.

SECTION 1

1 The use case, and why we need it

Use case statement (reconstructed)

An AI-assisted incident diagnosis for the HIP platform: **one timeline, ranked causes with evidence for and against, human-approved recovery**. Not detection (rules already do that), but compressing the 60 to 120 minutes between "alerts fired" and "we agree what broke."

Reconstructed from the implementation plan and our conversations; I have not seen the client's original written document. Replace with their wording if you have it.

The five outputs	What it means
1 Observed timeline	Ordered events, every row traceable to a source record; no inference
2 Ranked causes	2 to 4 hypotheses, confidence band, Supports and Contradicts evidence, "what would change this"
3 Diagnostic checks	Commands or queries ordered cheapest and most decisive first, with what each result means
4 Recovery proposal	Replay, repush, restart or rollback, with risk and rollback path, locked behind approval
5 Approval record	Who approved, when, what, on what evidence; nothing runs without it

Two non-negotiables (client requirements): facts and inferences never share a panel, and no recovery executes without a logged human approval.

Why it is needed

A warehouse cannot receive deliveries. Dozens of alerts fire from five systems, four duplicate tickets appear, the first-line runbook search returns generic matches, and a bridge call fills with engineers who each see only their own hop. **Every alert is correct. Nobody knows what broke.** The rules answered **what**; nobody has automated **why**.

	A monitoring rule	The AI layer
Answers	Is this over a threshold? Did this error appear?	What most likely caused all of this, and what do we check first?
Scope	One system, one signal	Every system and record at once
Reads	Metrics and known error strings	Also free text: vendor notices, tickets, change records
Absence	Only if a rule exists for exactly that	Notifies what did not happen
New situations	Only patterns someone anticipated	Reasons about combinations nobody anticipated
Acts alone?	Raises a ticket	No: a named human approves

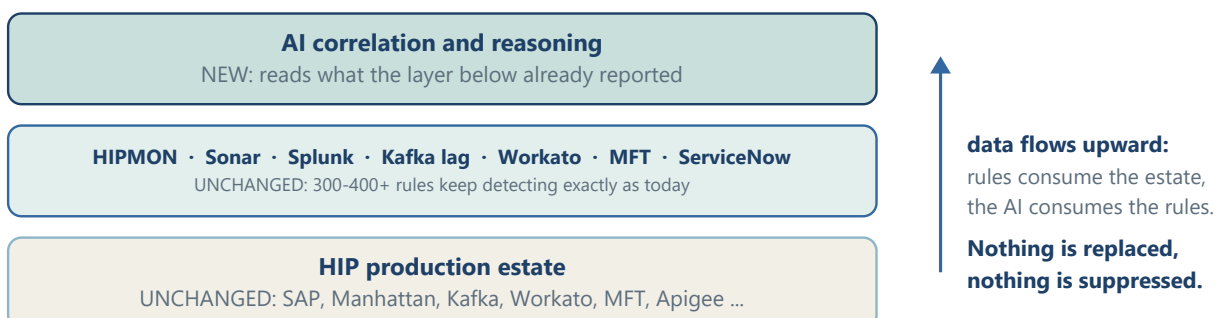


Figure 1. The AI is a new layer on top; everything below is unchanged and nothing is suppressed.

SECTION 2

2 How HIP works: one message, end to end

HIP is the client's integration platform: a postal service for business data between systems such as SAP, warehouse software and Salesforce.

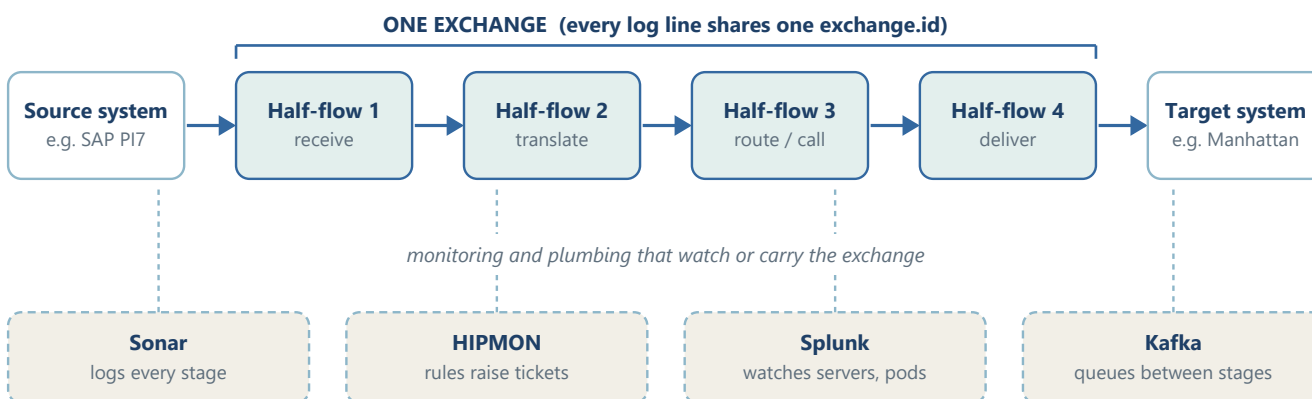


Figure 2. An exchange is a chain of half-flows; monitoring taps observe it.

Idea	Detail
Exchange	One business transaction end to end (like a distributed trace). It has an <code>exchange.id</code> shared by every log line.
Half-flow	One stage: receive, translate, route or call, deliver (like a span). Named like <code>EMEA_KEPLERNG_SALESORDER_OUT_02_ESB</code> .
halfflow.missing = 2/4	In our data, waiting at stage 2 of 4. Stalled, not failed, so no failure alert fires.
Statuses	COMPLETE, COMPLETE (F), INPROGRESS , WARNING, FAILED, REPLAYED. Log levels ENTRYINFO / EXITINFO / INFO / ERROR.
Alert format	<code>[AUTO] PROJECT HALFFLOW CODE ETP_CODE</code> (HIPMON); <code>CAPLINERR00x-Linux Host ...</code> (Splunk).
Correlation	The exchange.id, UTC timestamps (MFT emits local time with no offset), and three names for one thing (CMDB, half-flow, Kubernetes alias). Mismatches here are the classic reason correlation fails.

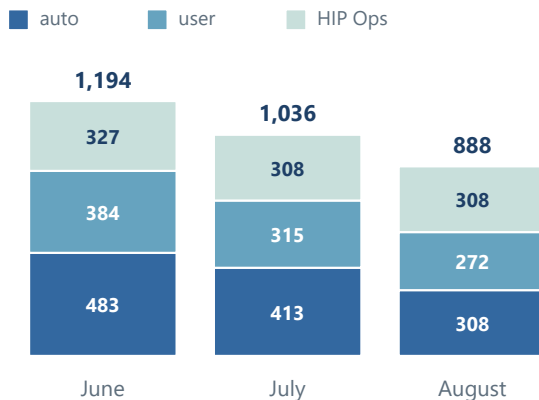
Layer	Components
Business systems	SAP S/4, SAP hubs CE / UK / IT / NE (via PI7), Manhattan warehouse system, Salesforce, Anaplan, 3PLs
Integration core	ESB (TIBCO half-flows on Kubernetes), Workato recipes, Apigee APIs, framework services such as <code>hip-fwk-transco-cache</code>
Queues and files	Confluent Kafka (one topic per processing group); MFT on Sterling File Gateway
Monitoring	Sonar (Kibana dashboards), HIPMON (rules raising <code>[AUTO]</code> tickets), Splunk (hosts and pods), 300 to 400+ rules
Ops tooling	ServiceNow (incidents, duplicates), change records, vendor tickets

SECTION 3

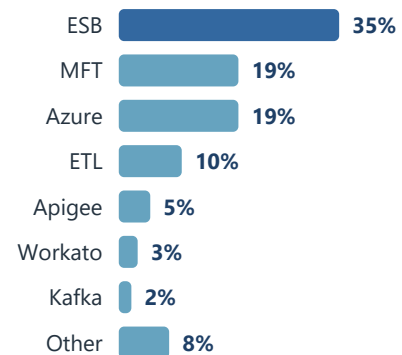
3 The real data we studied

Two real inputs shaped the design: **three Excel exports** of ServiceNow incidents (June, July, August 2026, one row per incident: subject, zone, priority, team, close code, root-cause notes) and **screenshots of the live Sonar "Full Levels" dashboard**. The exports hold employee names, so they stay in a git-ignored folder; the demo data only reproduces patterns with invented ids.

Incidents per month



Share of incidents by owning team (%)



Measure	Value	Why it matters
Auto-raised incidents	1,272 of 3,118 (41%); 868 [AUTO] across 190 half-flows	The alert layer works
Linked duplicates	390 incidents (12.5%)	One event raises several tickets
Median open-to-resolve	6.6 h auto-raised, 13.8 h user-raised	Ticket lifetime, not pure diagnosis time
Closed by replay	534 of 2,110 classified closes (25%)	Recovery is mostly "replay"; the gate must gate replay
Platform host alerts	406 (13%): swap, filesystem, CPU	The standing noise floor
Severity	25 Critical, 2 Major with written root causes	A ready-made test set
Where / when	PRD 82%; EMEA 59%; no quiet hours, peak 08:00 UTC	Demo runs on EMEA production

The five real incident families behind the scenarios

When (2026)	What happened	Root cause as the team wrote it	Became
Jun 4 to 5	ASNs missing in S4; 45k+ exchanges INPROGRESS	Heavy half-flow overloaded a processing group; moved to a bigger one, stuck orders re-pushed	S1
Jun 2	SAP to warehouse deliveries stopped; reprocessing did nothing	PI7 IDoc listener node hung; restart fixed it	S2
Jun 9, Jul 20	All IDoc outflows to SAP failing (ConfigForDocSending)	Vendor (Workato) push with missing parameters; emergency rollback to the legacy route	S3
Jun 10, 15	File exchange stuck INPROGRESS; database locked hours later	Connector failed in the EMEA patching window; replay fixed it	S4
Jul 28 to Aug 17	30+ unrelated flows fail: Connection refused ... transco-cache:8080	Shared cache pod unavailable; every replay then succeeded	S5

SECTION 4

4 Glossary: every term you will meet

Term	What it means
PLATFORM AND DATA UNITS	
HIP	The platform that moves business data between systems (a postal service for messages)
Exchange / half-flow	One business transaction / one stage of it (like a trace / a span)
exchange.id	ID shared by every log line of one exchange (like a trace ID)
halfflow.missing 2/4	Waiting at stage 2 of 4: stalled, not failed
Project	The business area a flow belongs to (e.g. SAPITCOMMON)
object.id / object.name	The business item moved (a delivery number; type such as DELVRY07)
event.code / event.reason	Log event type / the error reason text; identical reasons across flows point to one cause
STATUSES AND LOG LEVELS	
COMPLETE / COMPLETE (F)	Finished / finished after a failure and retry
INPROGRESS	Started, not finished. Raises no error, so silent stalls live here
FAILED / WARNING / REPLAYED	Errored / finished with a caveat / re-run by a person or job
ENTRYINFO / EXITINFO	Log lines written on entering and leaving a stage
MONITORING (THE "RULES" SIDE)	
Sonar	The client's name for their Kibana dashboards over exchange logs
Kibana / ELK / KQL	Log search and charts / Elasticsearch-Logstash-Kibana / the query language of the search box
HIPMON	The rule engine that raises [AUTO] ServiceNow tickets (868 in three months)
Splunk	Watches servers and pods (CPU, memory, disk, heap); alerts start CAPLINERR
Sonar hourly report	Scheduled summary of critical exchanges; counts a stall as InProgress, not Failed
Rule hit / context	An alert already fired / the records fetched around it (logs, metrics, changes). The AI reads both
INFRASTRUCTURE	
Kafka / Confluent	A durable message queue; producers write to a topic, consumers read it
Consumer lag	Messages written but not yet processed: a backlog
Processing group	Servers consuming one topic to execute half-flows
Kubernetes / pod	How software runs; a pod is one running copy of a service
Heap / GC	Working memory / its cleanup. A full heap makes GC run constantly ("GC thrash")
OOMKilled (exit 137)	A pod killed for exceeding its memory limit; it restarts
Health check	A probe reporting UP or DOWN; UP does not prove the service is doing work
INTEGRATION TOOLS	
ESB	The layer running the TIBCO half-flows; owns 35% of incidents
Workato / recipe	A vendor workflow platform; one workflow is a recipe
MFT / SFG	Managed file transfer / Sterling File Gateway
Apigee / APIM	API gateways; 5xx = server error, 4xx = client error
Control-M	A batch job scheduler
Transco cache	Shared service <code>hip-fwk-transco-cache</code> that many half-flows call

Term	What it means
SAP AND BUSINESS	
SAP S/4, hubs CE/UK/IT/NE	Main business software; the hubs look like regional SAP systems (inferred, not confirmed)
PI7	SAP's own integration hub, which sends messages into HIP
IDoc / RFC	SAP's document format / a remote function call into SAP (<code>ConfigForDocSending</code> is one)
ASN / STO	Advance shipping notice / stock transport order
Manhattan	The warehouse management system
OPERATIONS	
ServiceNow / INC	The ticketing system; <code>INC10790101</code> is an incident number
P1 to P5	Priority; P1 is worst (the data also has Major and Critical)
Parent / child	Several tickets for one event, linked (duplicates)
L1 / L2 / L3	Support tiers: first line, specialist, vendor or platform team
Bridge call	The live call where engineers work out what broke
Replay / repush	Re-run a failed exchange / re-send the data or file from the source
Change record (CHG)	Log of planned changes; CMDB is the inventory of systems and names
#01 to #04 notes	The ticket-closing format: Root cause, Resolution, Doc used, Related records
TERMS FROM THE AI BUILD	
Evidence pack	120 to 260 events chosen by code for the model to read
Supports / Contradicts	Evidence for and against a hypothesis, each citing an event id
Ruled out	Considered and dismissed, with the reason (unrelated alerts go here)
First symptom	Earliest event that belongs to the incident, including quiet precursors
Stored vs AI-live	Only S5 has a stored, hand-written answer; S1 to S4 are answered live by the model
Fallback	The last good live answer, saved and replayed if a call fails; labelled on screen

SECTION 5

5 The five scenarios

Five scenarios prove the system reasons rather than looks up an answer. **S1 and S2 are a pair**: the warehouse sees the same symptom ("deliveries not arriving") but the cause, evidence and first check differ. Each has a **planted contradiction**, a real record that argues against the right cause. Every scenario also contains two standing-noise alerts (a QA-host CPU alert and a swap alert) to be dismissed. Stored = the one hand-written answer (S5); AI-live = produced live by the model.

S1 Large-mapping heap AI-LIVE	
Fault	A 3,850-STO batch (04:52) makes ASN messages about 3x normal size; heap in one processing group fills, pods are OOMKilled, lag and INPROGRESS pile up.
Key signals	05:10 quiet heap alert (first symptom) · 06:15 ASNs stuck at 2/4 · 06:35 OOMKill · 07:30 hourly report shows 45,212 InProgress · 08:10 user ticket, trucks waiting.
Contradiction	Inflow is lower than last Thursday and the six other groups have zero lag: not load, not brokers; payload size .
Right answer	Oversized payloads exhaust heap (payload 40 to 121 KB in lockstep with heap). Check: message size vs heap. Fix: move flow to larger group, repush from S4.
Live result	Correct: payload first (Moderate), slow leak second (Low), 82,900 quoted from the evidence; the late record firmed it to High.

S2 PI7 listener hang AI-LIVE	
Fault	The listener receiving IDocs from SAP hub CE stops consuming but its process stays up. Nothing enters HIP , so there is nothing to replay.
Key signals	13:29:48 last exchange · 13:30:12 last heartbeat · inbound 0 vs baseline 180 · lag 0 on all 7 groups · rule fires only at 14:10 · replay matches 0 exchanges.
Contradiction	Health check says UP and heap/CPU are normal: it is up, just not consuming.
Right answer	Listener hung, no pile-up (unlike S1). First check: exchanges created in last 15 min. Fix: restart listener, SAP resends queued IDocs; not HIP replay.
Live result	Not confused with S1, but it hedges ("hub stopped sending" first, "listener hung" second). The late record (214 IDocs queued, threads blocked) then flips the ranking to the listener: a genuine flip.

S3 Vendor release AI-LIVE	
Fault	A Workato platform release changes a connector default; every recipe calling <code>ConfigForDocSending</code> fails. HIP has no change record .
Key signals	09:40 vendor notice · 09:52 first recipe failure · same error on hubs CE, UK, IT, NE · failed exchanges 39, 85, 188, 318 · 10:25 user ticket.
Contradiction	Some YHIPDELVR07 IDocs still succeed and the SAP gateway health is green: looks like a flaky SAP.
Right answer	Vendor regression 12 min before first failure. Check: search vendor notices before 09:52. Fix: emergency route to legacy path, then replay.
Live result	First run blamed SAP (our evidence had dropped that the failing step is a Workato call). After fixing that, two clean runs ranked the vendor release first.

S4 Stalled exchange AI-LIVE	
Fault	In the EMEA patching window the MFT connector drops. The 02:00 file arrives with status SUCCESS but its exchange stalls at 2/4 and never fails: no rule fires .
Key signals	02:00 file 4.1 MB vs 6.8 usual; declared 128,400 rows, present 77,032 · no half-flow-2 event · hourly reports show InProgress, Failed=0 · 07:57 database locked.
Contradiction	SFG says SUCCESS and every hourly report shows zero failures: the transfer "looked fine".
Right answer	Stall after the connector dropped in the patching window; no rule covers it . Fix: replay the exchange (approval-gated).
Live result	Correct on the first run, citing the missing half-flow and the row-count mismatch.

S5 Shared cache STORED	
Fault	The shared <code>hip-fwk-transco-cache</code> pod restarts (out of memory). For ~20 min every half-flow needing it fails, across unrelated projects.
Key signals	00:20 restart, exit 137 (first symptom) · 00:55 first <code>Connection refused ... :8080</code> · 30+ half-flows, 4+ projects · six tickets from six rules · 01:14 pod Ready, replays succeed.
Contradiction	By the time anyone looks the pod is Running and Ready and every replay works: looks like a network blip.
Right answer	One shared cause, the cache restart 35 min earlier. Fix: replay all failures in one batch (approval-gated).
Stored answer	The one hand-written answer, so this scenario never depends on the network. The live model reached the same cause once the prompt was tightened (first run promoted an unrelated mapping error and missed the restart).

SECTION 6

6 What we built

The governing principle: the AI sits on top of the rules

Its **input is rule output** (rule hits) plus retrieved **context**; it is never asked to find anomalies in raw streams. Where no rule fired, that absence is itself a finding. **Counts are code, not model**. Only S5 is stored (hand-written); S1 to S4 come from the live model. Nothing is fabricated to fill a gap.

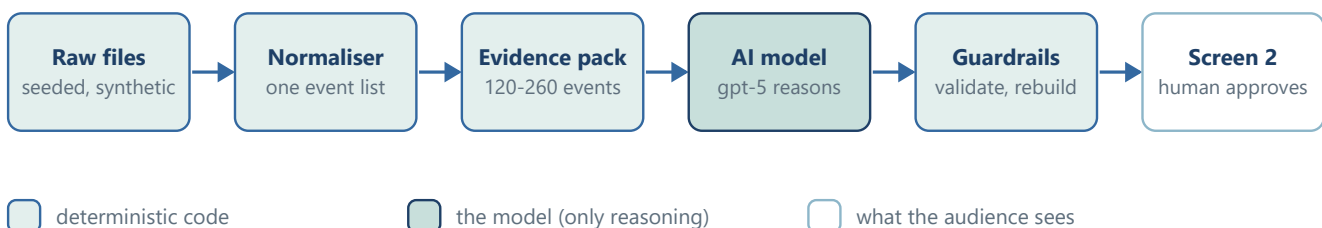


Figure 3. Facts are assembled by code and traceable to source records; only the reasoning about them is done by the model.

Step	What happens
Generate	Seeded synthetic raw files per scenario (Sonar logs, HIPMON alerts, Splunk, Kafka lag, Workato, Apigee, MFT, ServiceNow, changes) with 60 min of baseline and each contradiction as a real record. 44 checks against the raw files passed.
Normalise	One event list: UTC timestamps, MFT no-offset time resolved, aliases resolved, severity 1 to 4, exchange ids threaded; each event marked <code>rule_hit</code> or <code>context</code> .
Evidence pack	120 to 260 events: every distinct rule hit (repeats collapsed with counts), decisive baseline-vs-incident counts, bounded context. Absence is made explicit by code.
AI model	GPT-5 (Azure AI Foundry, reasoning effort low): hypotheses with evidence for and against, ordered checks, recovery proposal, draft ticket note.
Guardrails	Every citation checked (one retry, uncitable evidence dropped); JSON retry; timeline rebuilt by code ; approval forced on; rank diff computed by code; warnings for ungrounded numbers.
Fallback	Each successful live answer is saved and replayed if a later call fails, with a visible label. If none exists the screen says so.

Prompt rules the model must follow

Causes are hypotheses (2 to 4), each with supporting **and** contradicting evidence citing real event ids · checks ordered by decisiveness · recovery is a proposal · absence is evidence · **hypotheses are rival explanations of the same incident** (unrelated alerts go to Ruled out) · a change is a cause only if linked in time and component · first symptom includes precursors · count only the incident's own flows · High confidence only when the deciding check is done.

The app

- **Screen 1 "Today"**: a reproduction of the Sonar dashboard (working filters, KQL box, KPI tiles, exchange and trace tables), the HIPMON alert stream, the L1 panel and a bridge timer that keeps running.
- **Transition**: alert rows compress into evidence instead of vanishing, so the message is "consumed", not "replaced".
- **Screen 2 "With AI"**: A header, B observed timeline (fact), C candidate causes (inferred, dashed), D ruled out, E checks, F blast radius (fact), G recovery behind the approval gate. "Inject late evidence" shows the diagnosis firming up.

To run it	How
Live AI server	<code>cd server</code> then <code>npm start</code> (needs <code>server/.env</code> with the Foundry endpoint, key, model <code>gpt-5</code> , effort low); port 8787
App	<code>cd app</code> then <code>npm run dev</code> ; open http://localhost:5173
Rebuild data	<code>generate.py</code> , <code>normalise.py</code> , <code>find_anchors.py</code> , <code>build_alert_feed.py</code> , <code>build_bundles.py</code> , in that order

SECTION 7

7 The argument: "We already have 300 to 400 rules"

The answer in three sentences

"Your rules are the sensors and they work. Detection is not your problem; **diagnosis** is, the hours between the alerts and everyone agreeing what broke. Rules cannot close that gap because each one watches a single thing, and the answer lives in the relationship between them."

What rules cannot do	Proof in the demo
Correlate across systems	S5: 30+ correct alerts and six tickets from one cache restart no rule watched
Notice absence	S4: five hours INPROGRESS, no rule fires. S2: nothing arrives; the only rule fires after 60 min
Read unstructured text	S3: a vendor notice 12 minutes before the first failure
Cover novel combinations	A rule needs someone to have predicted the pattern first
Argue against themselves	Every AI cause lists contradicting evidence and what would change it
Order next steps, draft the note	Checks ordered by decisiveness; #01 to #04 ticket note drafted

What the AI does not do (say this openly)

- It does not replace any rule or reduce rule maintenance; the rules are its input.

- **It does not act alone.** A named person approves every recovery and the approval is logged.
- **It can be wrong.** We saw a wrong answer on one run; that is why the guardrails exist.
- **It needs good signals:** consistent exchange ids, timestamps and naming.

State this before they ask

The demo uses **synthetic incidents patterned on the real data**, not real replays. Accuracy on real incidents has **not** been measured yet: that is exactly what the proposed next step measures.

They say	You say
"We have 400 rules."	Agree. "They all fired correctly: that is the first screen. The rules are the sensors, this is the analyst."
"Why not correlation rules?"	"Someone must anticipate each pattern first, and a rule cannot read a vendor notice or notice what did not happen."
"Can we trust an AI?"	"It cannot execute anything. It cites evidence, shows what argues against itself, and a named human approves."
"This is synthetic data."	"Yes, built from your own error strings and patterns. Give us one real past P1 and we will run it."
"Does it cut rule maintenance?"	Do not oversell: "No. It changes what happens after rules fire."

The pitch

1. **"Your 400 rules are the sensors. This is the analyst."**
2. **"Detection is solved and working; diagnosis is where the hours go, and rules cannot do it."**
3. **"Give us one real past P1 and we will run it. It costs nothing and needs no production access."**

The question that usually settles it

"On your last cross-system P1, how long was it between the first alert and everyone agreeing on the cause?"

Next step to propose: retrospective validation on real past P1s (the export has 27 Critical or Major with written root causes), then shadow mode alongside live incidents. A real deployment first needs a signal-readiness check: exchange-id coverage, timestamps and name mapping.

SECTION 8

8 Findings: what worked, what did not

- **Worked:** the pipeline is deterministic (identical hashes on re-run); decisive evidence is real data (44 raw-file checks passed); The stored S5 answer validates (16 citations resolved); the live AI reached the right cause on S1, S3, S4 and S5 after fixes, and S2 hedges until the late record settles it; guardrails held; both UI paths ran in a real browser without errors.

Problem	Cause and fix	Status
AI promoted an unrelated alert; missed the first symptom (S5)	Prompt did not scope hypotheses to one incident. Added rival-explanation, change-linking and first-symptom rules	Fixed
S3 gave a worse answer on a later run	Our evidence had dropped that the failing step was a Workato call. Fixed the evidence, added count and citation rules	Fixed
493 "stuck" reported instead of 318	It was the in-progress count. Prompt now says use the right status; server flags ungrounded numbers	Fixed

Problem	Cause and fix	Status
Live latency 124 s	Reasoning effort low (about 75 s) and background prefetch	Improved
Malformed JSON broke a call	Server retries once on a parse error	Fixed
Screen 1 chrome inert; columns squashed	Working filters, KQL, tabs; percentage column widths	Fixed
Stored answers cannot flip honestly	The evidence supports one cause from the start; a stored flip would be staged. Live S2 now flips for real when the late record arrives	Resolved

The lesson from S3

When the AI is wrong, **check the evidence you gave it before blaming the model**. The first S3 answer was a fair reading of contradictory evidence.

Known limitations

- Live calls take 70 to 80 s and can hit a **rate limit** if fired together; do not switch quickly between AI-only scenarios, and run each once before a demo so a fallback exists.
- GPT-5 has no fixed seed, so run-to-run variation remains; two clean runs is a small sample.
- Scenarios are patterns from real incidents, not replays; real accuracy is unmeasured.
- The saved fallbacks are git-ignored, so a fresh clone has none until each scenario has run once.

SECTION 9

9 Revision

Flashcard	Answer
What is an exchange / half-flow?	One business transaction end to end (a trace) / one stage of it (a span).
Why is INPROGRESS dangerous?	It is not an error, so a stalled exchange raises no failure alert.
What does halfflow.missing 2/4 mean?	Waiting at stage 2 of 4: stalled, not failed.
Rule hit vs context?	An alert already fired vs the records fetched around it.
What is consumer lag?	Messages written to Kafka but not yet processed: a backlog.
S1 in one line?	An oversized batch exhausted heap in one processing group; lag and INPROGRESS pile up.
S2 in one line?	The PI7 listener hung: zero inflow, zero lag, flat heap, nothing to replay.
How do S1 and S2 differ?	S1 is a pile-up; S2 is no inflow. Same symptom, different first check.
Which scenario is stored?	Only S5 (hand-written). S1 to S4 are answered live; the last good live run is the fallback.
S3 in one line?	A vendor Workato release broke every recipe calling ConfigForDocSending; no HIP change record.
S4 in one line?	A patching window dropped a connector; the file said SUCCESS but its exchange stalled and no rule fired.
S5 in one line?	A shared cache pod restarted; 30+ unrelated flows failed with one error.
Why does the AI not detect?	The rules already detect; the AI explains what they collectively mean.
What is "facts by code"?	The timeline is built from source records by code; the model only reasons about them.

Flashcard	Answer
Why require a Contradicts column?	An empty column looks like assertion; it forces honest weighing.
What gates recovery?	A named approver with a reason, logged before anything runs.
Best next step to propose?	Run one real past P1 and compare with what the team concluded.

Quiz

#	Question	Answer
1	Why is "41% auto-raised" an argument for the AI?	It shows detection works; the gap is diagnosis, which rules cannot do.
2	Why not say "diagnosis takes 6.6 hours"?	It measures how long tickets stay open, waiting and hand-offs included.
3	In S2, why does replay not help?	Nothing entered HIP, so there is nothing failed or stuck to replay.
4	In S4, why did no alert fire?	The exchange stayed INPROGRESS, SFG said SUCCESS, and reports count it as InProgress.
5	In S5, what was the first symptom?	The cache pod restart at 00:20, 35 minutes before the first failure.
6	Why did the first live S3 run go wrong?	Our evidence hid that the failing step was a Workato call.
7	Which outputs are computed by code, not the model?	Timeline, rules-fired counts, rank diff, forced approval, citation checks.
8	What is the strongest next step?	One real past P1: run it and compare. It costs them nothing and needs no production access.

The 60-second pitch

"You already have 300 to 400 rules and they work: forty-one percent of your incidents in the last three months were raised automatically. The trouble is what happens next. One event raises several tickets, five teams each see their own hop, and tickets stay open for hours while people work out what actually broke.

Rules answer **what** is wrong. Nobody has automated **why**. This layer reads what your rules already fired, plus the logs, host metrics and change records around them, and gives you one timeline, ranked causes with evidence for and against, the check to run first, and a recovery a named person approves. It never replaces a rule and never acts alone.

What you saw is built from your own patterns but on synthetic incidents, so the honest next step is small: give us one real past P1 and we will run it and compare with what your team concluded."

3,118 incidents	868 [AUTO] tickets	27 Critical or Major (test set)	25% closed by replay	75 s live AI at low effort
---------------------------	------------------------------	---	--------------------------------	--------------------------------------